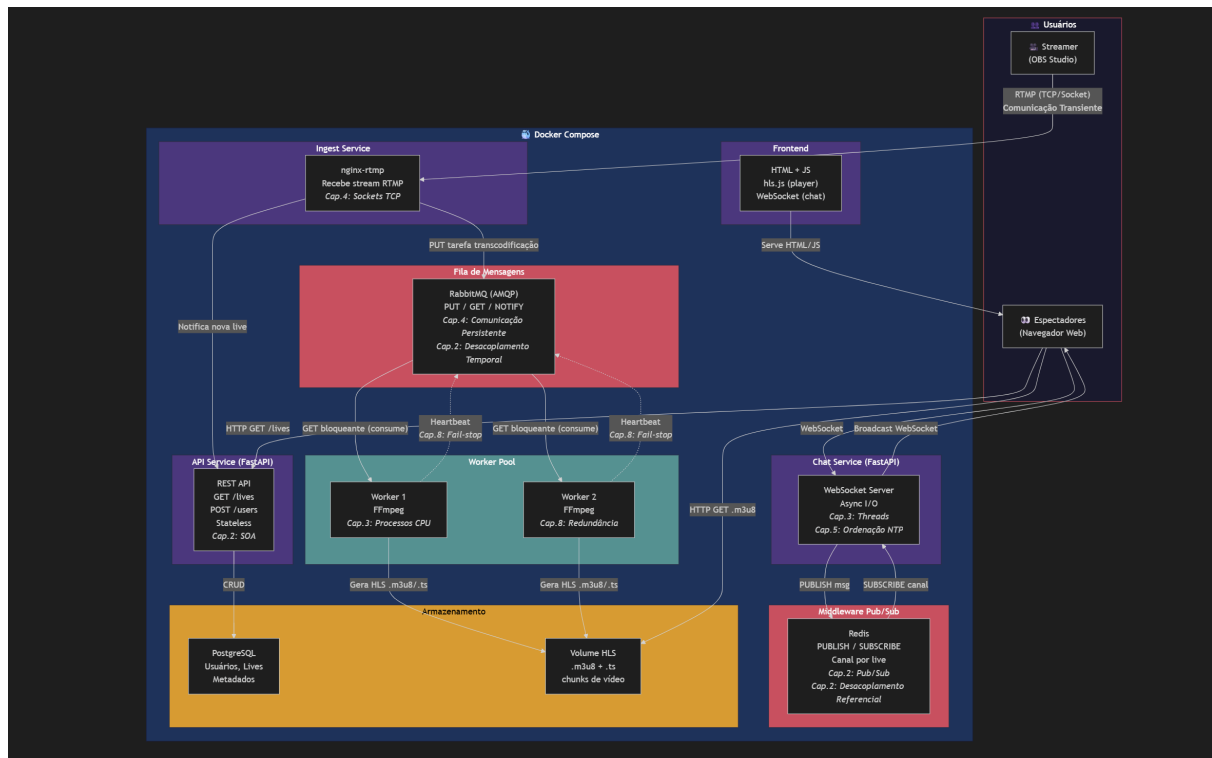
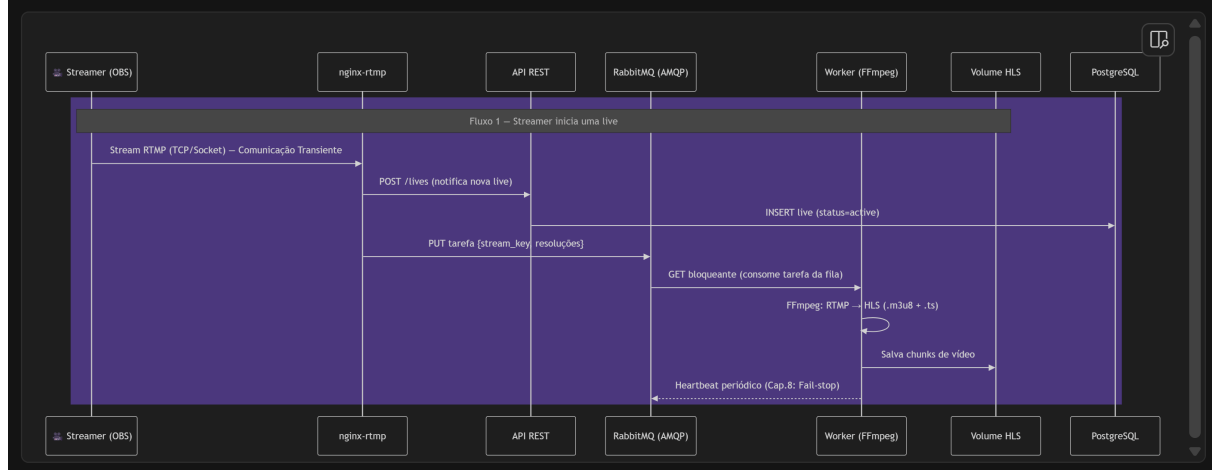
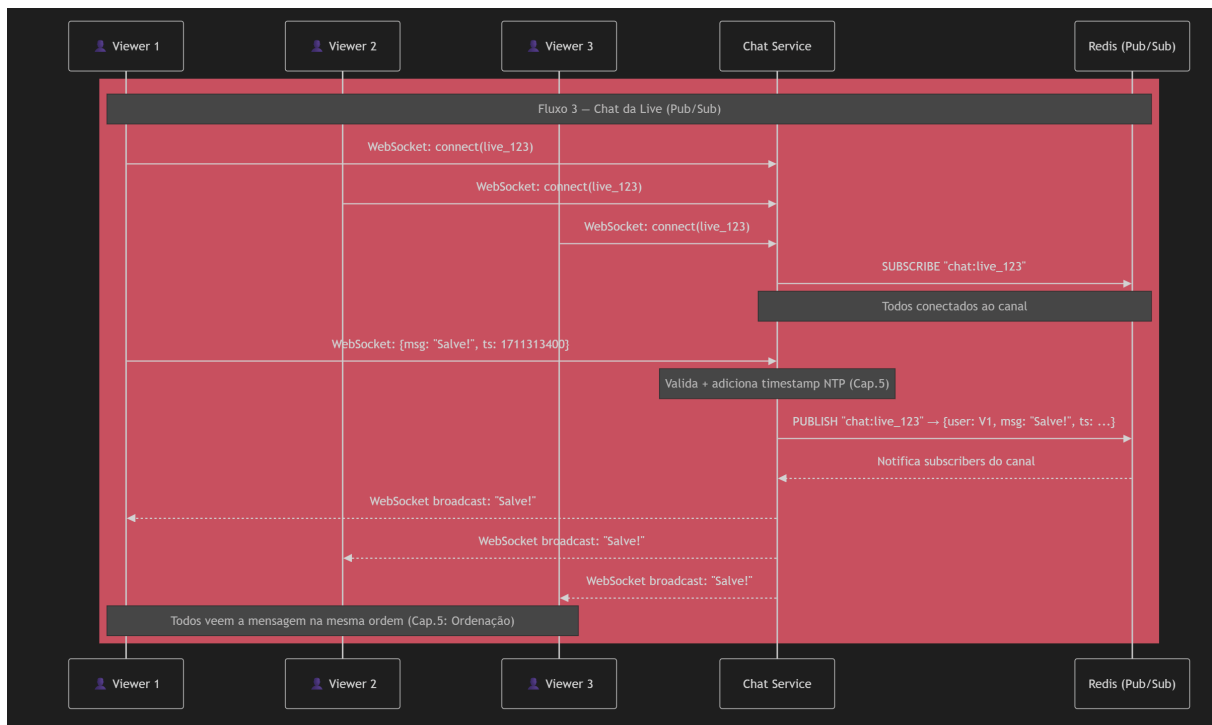
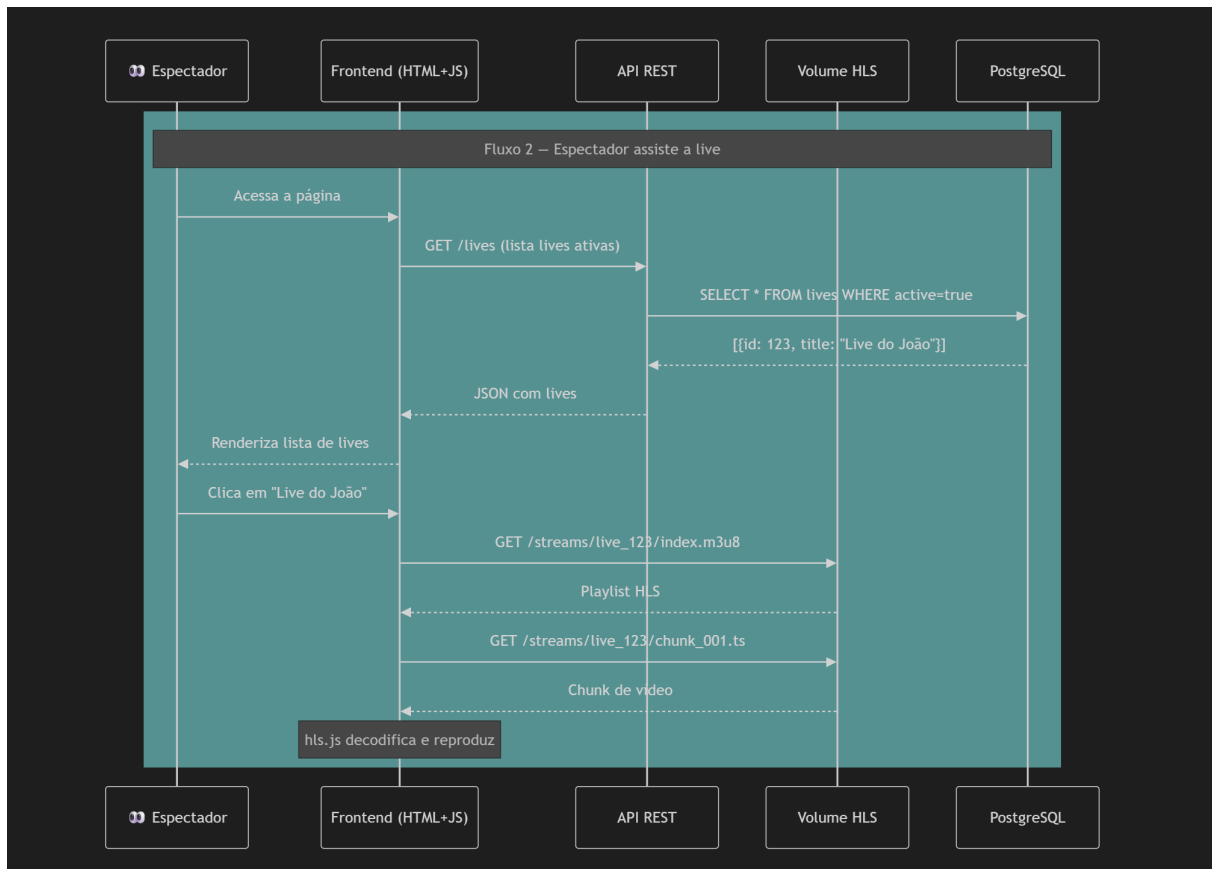


distribuidos

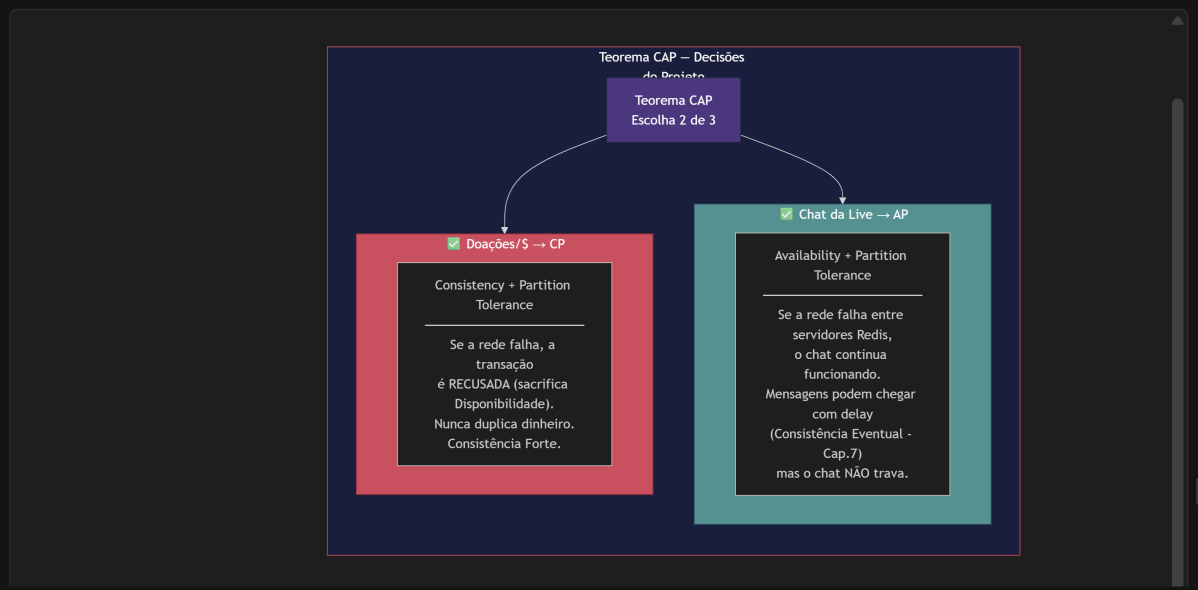


Agora o diagrama de sequência dos 3 fluxos principais:





E por último, o diagrama de decisão do Teorema CAP:



Pronto. Aqui estão os 4 diagramas Mermaid:

1. **Arquitetura geral** — todos os componentes, conexões e referências aos capítulos
2. **Fluxo 1** — Streamer inicia live (RTMP → nginx → RabbitMQ → Worker → HLS)
3. **Fluxo 2** — Espectador assiste (API REST → HLS chunks → player)
4. **Fluxo 3** — Chat Pub/Sub (WebSocket → Redis PUBLISH/SUBSCRIBE → broadcast)
5. **Teorema CAP** — decisões AP (chat) vs CP (doações)

Os "servidores" (containers) são 7:

#	Container	O que faz	Está ligado o tempo todo
1	nginx-rtmp	Recebe o vídeo do OBS	Sim
2	API	CRUD de lives/usuários (REST)	Sim
3	Chat	WebSocket + publica no Redis	Sim
4	Redis	Middleware Pub/Sub (repassa msgs)	Sim
5	RabbitMQ	Fila de tarefas AMQP	Sim
6	Worker (1 ou mais)	FFmpeg (transcodifica vídeo)	Sim, consumindo a fila
7	PostgreSQL	Banco de dados	Sim

Os 3 fluxos cruzam esses containers:

```
FLUXO 1 (iniciar live):  nginx → API → PostgreSQL
                        nginx → RabbitMQ → Worker → gera HLS

FLUXO 2 (assistir):      Frontend → API → PostgreSQL
                        Frontend → busca arquivos HLS que o Worker gerou

FLUXO 3 (chat):          Frontend → Chat → Redis → Chat → Frontend
```

```
Fluxo 1 passa por [nginx] → API → PostgreSQL
                  → RabbitMQ → Worker

Fluxo 2 passa por [Frontend] → API → PostgreSQL
                  → Arquivos HLS

Fluxo 3 passa por [Frontend] → Chat → Redis → Chat → Frontend
```

O ponto chave:

- API e PostgreSQL são usados nos fluxos 1 e 2
- Redis só é usado no fluxo 3
- RabbitMQ e Worker só são usados no fluxo 1
- nginx-rtmp só é usado no fluxo 1

Todos os containers rodam **ao mesmo tempo** numa mesma máquina (a sua), orquestrados pelo `docker-compose up`. Na perspectiva do professor, cada container é um processo **independente**, potencialmente em máquinas diferentes — e é exatamente isso que faz ser um **sistema distribuído**: vários processos cooperando via rede (REST, WebSocket, AMQP, Pub/Sub).

como fazer de fato

anotações pascale

Sim, é exatamente isso! Mas não é banco de dados — é uma **pasta no disco** (volume Docker). Funciona assim:

O que é o "Volume HLS"

É literalmente uma **pasta compartilhada** entre containers:

No Docker Compose você define:

```
volumes:
  hls_data: /tmp/hls      ← pasta real no disco
```

Containers que enxergam essa pasta:

- Worker: ESCREVE arquivos nela
- nginx: SERVE esses arquivos via HTTP (porta 8080)

```
Disco do computador
└─ /tmp/hls/
   └─ minha_stream_key/
      ├── 720p/
      │   ├── index.m3u8      ← playlist (arquivo de texto)
      │   ├── chunk_001.ts    ← 2s de vídeo (~500KB)
      │   ├── chunk_002.ts    ← 2s de vídeo
      │   ├── chunk_003.ts    ← 2s de vídeo (mais recente)
      │   └─ (mais antigos são apagados automaticamente)
      └─ 480p/
          ├── index.m3u8
          ├── chunk_001.ts
          └─ ...
```

O ciclo completo passo a passo

PASSO 1: Worker escreve no disco (a cada ~2 segundos)

```
Worker (FFmpeg) recebe RTMP do nginx
├─ Acumula 2 segundos de vídeo
├─ Salva como arquivo: /tmp/hls/abc123/720p/chunk_047.ts
└─ Atualiza a playlist: /tmp/hls/abc123/720p/index.m3u8
    (adiciona chunk_047, remove chunk_044 que já é velho)
```

PASSO 2: nginx serve a pasta via HTTP

nginx tem uma config simples:

```
location /hls/ {
    root /tmp;    ← aponta pra mesma pasta
}
```

Então qualquer GET vira uma leitura de arquivo:

```
GET <http://localhost:8080/hls/abc123/720p/index.m3u8>
→ nginx lê /tmp/hls/abc123/720p/index.m3u8 do disco
→ retorna o conteúdo
```

PASSO 3: Frontend (hls.js) fica pedindo

hls.js no navegador faz automaticamente:

a cada ~2s:

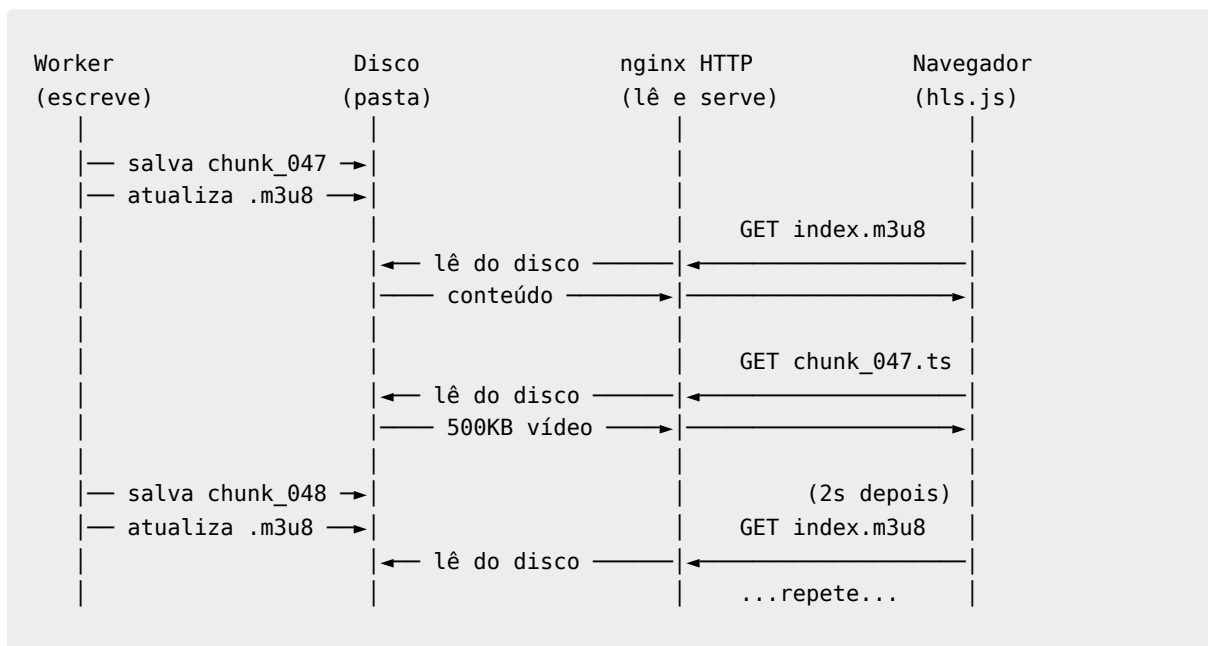
```
GET /hls/abc123/720p/index.m3u8 → "quais chunks existem?"

resposta (arquivo de texto):
#EXTM3U
#EXT-X-TARGETDURATION:2
#EXTINF:2.0,
chunk_045.ts
#EXTINF:2.0,
chunk_046.ts
#EXTINF:2.0,
chunk_047.ts      ← novo!

hls.js vê que chunk_047 é novo:
GET /hls/abc123/720p/chunk_047.ts → baixa 500KB de vídeo

decodifica e mostra na tag <video>
```

Visualização do fluxo



Então respondendo direto:

O worker guarda em banco de dados?

Não. Guarda em **arquivos no disco** (pasta compartilhada via Docker volume).

O front fica dando GET?

Sim! A biblioteca **hls.js** faz isso automaticamente. Você só precisa de:

```

<video id="player"></video>
<script src="hls.js"></script>
<script>
    var hls = new Hls();
    hls.loadSource('<http://localhost:8080/hls/abc123/720p/index.m3u8>');
    hls.attachMedia(document.getElementById('player'));
    // hls.js faz os GETs sozinho a partir daqui
</script>

```

Você escreve **6 linhas de código** e o hls.js cuida de todo o ciclo de GET → decodificar → mostrar no player.

